

Dino Game Bot — Deep Dive

A pixel-vision autoplayer for Chrome’s offline Dino game, with a live status dashboard

Explainer document generated for the project owner

This document explains the project exhaustively for someone who has never seen the code before. Every technical term is defined the first time it is used. It is a working reference for the owner, not published website content.

Contents

1	What this project is	2
1.1	Why it exists (from the README)	2
2	Prerequisite concepts, defined before use	2
3	High-level architecture	4
4	Module-level constants	4
5	Detection logic	5
5.1	Why day/night mode matters	5
5.2	<code>check_day()</code>	5
5.3	<code>check_obstacle(is_day=None)</code>	5
5.4	Decision loop (inside <code>main()</code>)	6
6	Status dashboard	6
6.1	Design goal	6
6.2	<code>StatusTracker</code> : display-independent state	6
6.3	Two views, one interface	7
6.4	<code>build_status_view()</code> : choosing a view safely	8
6.5	Sequence of one loop iteration, end to end	8
7	Directory layout	9
8	Dependencies and their roles	9
9	Setup and usage, as documented	9
10	Documented challenges and design trade-offs	9
11	What was verified vs. not (per the README)	10
12	License	10

1 What this project is

The Dino Game Bot is a single Python script (`main.py`) that plays Chrome’s built-in offline dinosaur game automatically. That game (reachable in Chrome by navigating to `chrome://dino/`, or by going offline and hitting a network error page) shows a small pixel-art dinosaur running across the ground; the only possible input is “jump” (the space bar), and the only way to lose is to let an obstacle (a cactus or a bird) collide with the dinosaur.

The bot does not read the game’s internal state, its DOM (**DOM** = Document Object Model, the browser’s in-memory tree representation of a web page’s elements — the bot never inspects this tree to find game objects), or any JavaScript variables. It only looks at the same thing a human player would see: pixels on the screen. It decides whether to jump purely from the colors present in two fixed rectangular regions of the screen, captured repeatedly in a tight loop.

On top of that detection loop, a second concern – a **status dashboard** – was added afterward: the bot has no visual output of its own beyond the Chrome window it drives, so a small live status window (or, when no graphical display is available, printed console lines) surfaces what the detection loop is currently seeing and doing, once per loop iteration.

1.1 Why it exists (from the README)

The Dino game has exactly one input and a simple visual language, which makes it a small, self-contained target for practicing screen-based automation: no game API to call, no memory to read, just pixels on screen and a keyboard event. The project exercises two different tools together: Selenium (to drive Chrome) and raw screen capture / pixel inspection (to “see” the game), rather than solving the whole problem through a single library.

2 Prerequisite concepts, defined before use

Screen capture / screenshot region

Reading the pixel colors currently displayed on the physical screen, for a given rectangular area, without any cooperation from the application being read. The bot uses this to see the game the same way a human eye would, without any access to what Chrome “knows” internally.

Pixel / RGB tuple

A pixel is one colored dot in an image. Its color here is represented as an **RGB tuple**, three integers 0–255 for red, green, and blue channel intensity, e.g. (255, 255, 255) is pure white and (83, 83, 83) is a dark grey. Two pixels are “the same color” if all three numbers match exactly.

Bounding box (bbox)

A rectangle described by four numbers: (`left`, `top`, `right`, `bottom`), each an absolute pixel coordinate measured from the top-left corner of the screen. Screen-capture libraries use this to know which rectangular slice of the screen to read.

Selenium / WebDriver

A browser-automation library. It lets a script open a real browser, navigate to URLs, find page elements, and send keyboard/mouse events to them, as if a human were operating the browser. **WebDriver** is the underlying protocol/API Selenium implements to talk to the browser.

ChromeDriver

A small helper program that Selenium launches alongside Chrome; it is the actual bridge between Selenium’s commands and the Chrome browser process. Selenium 4.6+ ships **Selenium Manager**, which downloads and locates a matching ChromeDriver automatically, so a manual driver install is usually unnecessary.

Polling loop

A loop that repeatedly checks (“polls”) some condition on a fixed cadence, rather than being notified asynchronously when something changes. The bot’s `main()` loop is a polling loop: every iteration it re-captures the screen and re-evaluates whether to jump.

tkinter

Python’s standard-library GUI (Graphical User Interface) toolkit, used here to build the small status window. It requires a graphical display to be available (an X server on Linux, or the native windowing system on macOS/Windows); it cannot run in a purely text-only/headless environment.

Headless

Describes an environment with no graphical display attached (e.g. a server or a CI container). Code that assumes a screen/GUI is available will typically fail to even construct a window in such an environment.

Non-blocking update loop

Most GUI toolkits expect the program to hand control over to them via a “main loop” (`mainloop()` in tkinter) that runs forever and processes window events. That would prevent the bot’s own detection loop from ever running. Instead this project calls a manual, non-blocking refresh (`update_idletasks() + update()`) once per detection-loop iteration, so the GUI and the detection loop share the same thread without either blocking the other.

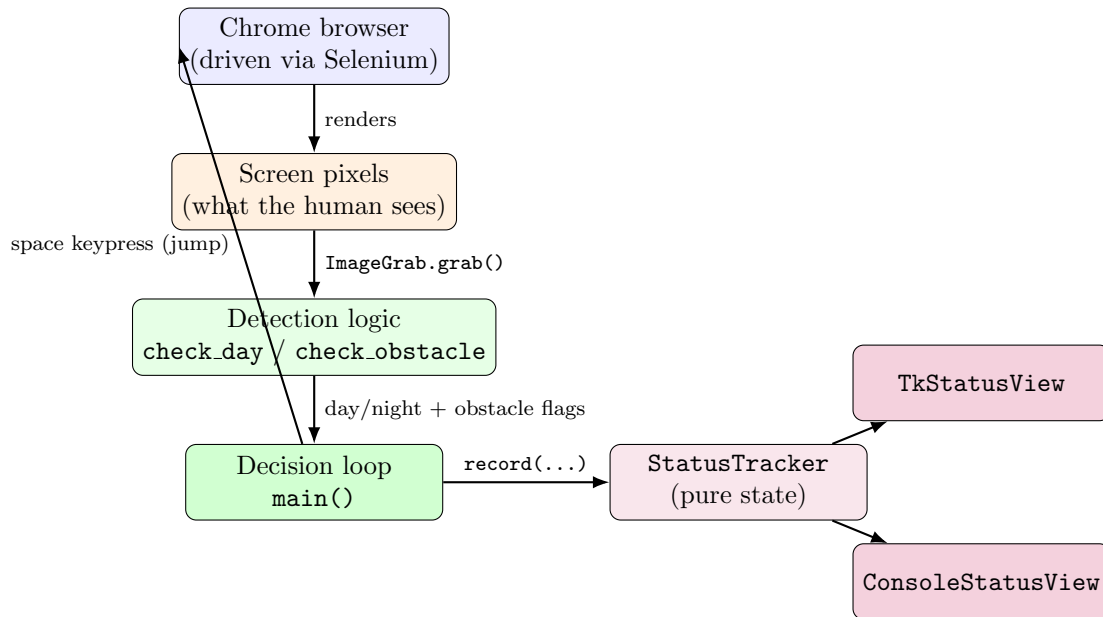
Deque with maxlen

A **deque** (double-ended queue) is a list-like structure optimized for adding/removing from both ends. One created with `maxlen=N` silently drops the oldest item whenever a new one is appended past N items — an automatically size-capped rolling log, used here to keep only the most recent 8 status events.

Monotonic clock

A clock that only ever moves forward and is unaffected by the system’s wall-clock time being changed (e.g. by NTP sync or the user changing the date). Used for measuring elapsed durations reliably, as opposed to `datetime.now()`, which is used only for human-readable timestamps in the log.

3 High-level architecture



Two independent concerns share the same loop:

1. **Detection and play:** capture pixels, decide day/night mode, decide if an obstacle is present, and press space if so. This is the original logic and is entirely self-contained — it does not know the dashboard exists.
2. **Status reporting:** after each detection step, the loop hands the raw results to a small state object (**StatusTracker**), which is then displayed by whichever view was selected at startup (tkinter window, or a console fallback). The dashboard is purely an observer; it cannot influence whether the bot jumps.

4 Module-level constants

Name	Meaning
OBSTACLE_AREA.COORDINATES	(380, 645, 950, 760) — the narrow bounding box positioned just in front of the dino, where an obstacle pixel means “jump now.”
GAME_AREA.COORDINATES	(20, 580, 800, 760) — a larger bounding box covering ground and sky, used only to decide day vs. night mode by counting pixel colors.
DAY_GROUND.COLOR	(255, 255, 255), pure white — the background color in day mode.
NIGHT_GROUND.COLOR	(83, 83, 83), dark grey — the background color in night mode.
STATUS_LOG.SIZE	8 — how many recent status events the dashboard keeps in its rolling log.
CONSOLE_STATUS.INTERVAL	1.0 seconds — minimum gap between console status prints in headless mode, so the fallback doesn’t spam a line on every ~10 ms loop iteration.

Both coordinate boxes are *absolute screen pixels*, tuned for one specific maximized Chrome window on one specific screen resolution. They are not derived from the browser window’s actual position,

so the bot only works correctly on the exact setup it was measured against; the README explicitly warns that they need re-measuring on any other machine.

5 Detection logic

5.1 Why day/night mode matters

The Dino game periodically inverts its color palette: in *day* mode the sky and ground are light (background approaches white) and obstacles render as dark grey sprites; in *night* mode the palette flips (background approaches dark grey, obstacles render as light/white sprites). A detector that only ever looks for “dark pixels = obstacle” would work in day mode and then silently stop working the moment the game flips to night mode. So the bot must first figure out which mode it is currently in, and only then decide which color it is hunting for in the obstacle box.

5.2 check_day()

```
1 def check_day():
2     image = ImageGrab.grab(bbox=GAME_AREA_COORDINATES)
3     pixels = list(image.getdata())
4     return pixels.count(DAY_GROUND_COLOR) > pixels.count(
        NIGHT_GROUND_COLOR)
```

`ImageGrab.grab` (from the Pillow / **PIL** image library — PIL stands for Python Imaging Library, Pillow is its actively maintained successor) takes a screenshot restricted to the given bounding box and returns an in-memory image object. `image.getdata()` flattens that image into a plain list of pixel tuples, one per pixel, in row-major order (left to right, top to bottom). The function then simply counts how many pixels are exactly pure white versus exactly the night grey, across the whole `GAME_AREA` box, and returns `True` (“it is day”) if white pixels outnumber grey ones. This is a majority vote over a large area that mostly contains background (sky/ground), so a handful of obstacle or dino pixels of the “wrong” color do not skew the result.

5.3 check_obstacle(is_day=None)

```
1 def check_obstacle(is_day=None):
2     image = ImageGrab.grab(bbox=OBSTACLE_AREA_COORDINATES)
3     pixels = list(image.getdata())
4     if is_day is None:
5         is_day = check_day()
6     if is_day:
7         return NIGHT_GROUND_COLOR in pixels
8     return DAY_GROUND_COLOR in pixels
```

This grabs the smaller, closer `OBSTACLE_AREA` box — the “jump trigger zone” just ahead of the dino — and checks for the presence (`in pixels`, a membership test, true if *any* pixel matches) of the color that represents an obstacle in the current mode. In day mode, obstacles are dark grey against a white background, so it looks for `NIGHT_GROUND_COLOR` (`(83,83,83)`) — confusingly named from the ground-color constant it reuses, but in day mode that same dark grey value is what an obstacle sprite renders as. In night mode the search target flips to white. A single matching pixel anywhere in the box is enough to report an obstacle; there is no size or shape check.

The `is_day` parameter is an optimization added alongside the dashboard: the main loop already needs to know the current mode for its own purposes (to display it and to feed `StatusTracker`), so it computes `check_day()` once per iteration and passes the result in, avoiding a second, redundant screen capture. If the caller omits it, the function computes its own mode exactly as the original, dashboard-free version did — this is also the code path exercised when the function is tested in isolation.

5.4 Decision loop (inside `main()`)

```
1 while True:
2     is_day = check_day()
3     obstacle = check_obstacle(is_day)
4     jumped = False
5     if obstacle:
6         driver.find_element(value="t").send_keys(Keys.SPACE)
7         jumped = True
8         time.sleep(0.01)
9     tracker.record(is_day, obstacle, jumped)
10    status_view.update(tracker)
11    if getattr(status_view, "closed", False):
12        break
13    if keyboard.is_pressed("q"):
14        break
```

There is no distance or speed estimate anywhere in this project: the obstacle box is placed close enough to the dino that “an obstacle-colored pixel is present in that box” and “jump immediately” are treated as the same event. `driver.find_element(value="t")` looks up the page element whose `id` attribute is “t” (the Dino game’s own runner container element) and sends it a space keydown, which the game interprets as a jump regardless of which element technically receives focus. After a detected jump the loop sleeps 10ms before its next check, trading a small amount of extra CPU polling frequency for faster reaction time. The loop exits either when the status window is closed by the user or when the `q` key is held/pressed (checked via the `keyboard` library’s global hotkey polling, which on Linux needs elevated privileges because it reads the input device directly rather than going through the browser).

6 Status dashboard

6.1 Design goal

The bot’s only native output is the Chrome window it drives; nothing in the original detection logic reports what it is currently “thinking.” The dashboard closes that gap: once per loop iteration, whatever the loop just observed (mode, obstacle presence, whether it jumped) is pushed into a small state object, which any display layer can read and render, without the detection logic needing to know or care whether a window exists at all.

6.2 StatusTracker: display-independent state

```
1 class StatusTracker:
2     def __init__(self, log_size=STATUS_LOG_SIZE, clock=time.monotonic):
3         self._clock = clock
4         self.start_time = self._clock()
```

```

5         self.is_day = None
6         self.obstacle_detected = False
7         self.jump_count = 0
8         self.log = deque(maxlen=log_size)

```

`StatusTracker` holds no reference to `tkinter`, the console, or anything display-related: it is a plain Python object, which is what makes it possible to unit-test the dashboard's *logic* without a screen, a browser, or a GUI toolkit at all. Its constructor accepts the clock function as a parameter (**dependency injection** — passing in a collaborator, here the clock, rather than hardcoding a call to it, so a test can substitute a fake, fully controllable clock).

`record(is_day, obstacle_detected, jumped, timestamp=None)` is called once per loop iteration with the loop's raw results. It:

- detects a *mode flip* by comparing the new `is_day` to the previously stored one (and treats the very first call, where the old value is `None`, as not a flip);
- increments `jump_count` if `jumped` is true;
- builds a human-readable event string: "jump", or "obstacle" (detected but a jump wasn't recorded as sent that iteration in this path), or "clear", with " (mode flip)" appended when applicable;
- appends a dict with time, mode, and event text to the capped `log` deque, evicting the oldest entry once past 8 entries;
- returns the appended entry, mainly so tests can assert on it directly.

`snapshot()` returns a plain dict of everything a view needs to render (mode as a string, obstacle flag, jump count, elapsed seconds via `elapsed_seconds()`, and a materialized list copy of the log) so views never touch `StatusTracker`'s internals directly.

6.3 Two views, one interface

Both view classes expose the same two-method shape — `update(tracker)` and `close()` — even though nothing in the codebase formally declares that as a shared interface (Python does not require one; this is convention enforced by usage, sometimes called **duck typing**: if it has the methods you call on it, it works, regardless of its declared type).

`TkStatusView` builds a small always-present `tkinter` window: a title bar, four live text labels (mode, obstacle, jump count, elapsed time, each backed by a `tk.StringVar` so updating the variable updates the label automatically), and a `Listbox` showing the rolling event log. Critically, its `update()` method never calls `tkinter`'s own `mainloop()` (which runs forever and would never give control back to the detection loop); instead it calls `root.update_idletasks()` (recompute pending layout/geometry) followed by `root.update()` (process any pending window-system events, such as the close button), then returns immediately. This is what allows the GUI and the ~10 ms polling loop to coexist on a single thread. Clicking the window's close button does not destroy the window itself; it only flips an internal `_closed` flag via the `WM_DELETE_WINDOW` protocol handler, which `main()` checks after every `update()` call to decide whether to stop the whole bot.

`ConsoleStatusView` is the headless fallback: it stores the last time it printed and, on each `update()` call, only actually prints a new status line if at least `CONSOLE_STATUS_INTERVAL` (1 second) has elapsed

since the last print. Without that throttle, a console fallback driven from a ~ 10 ms loop would flood stdout with dozens of lines per second.

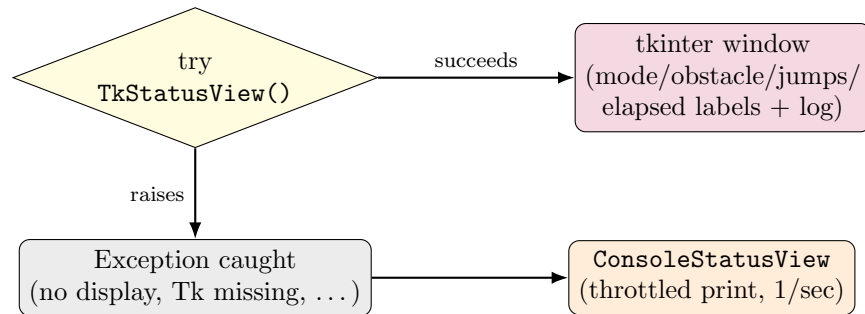
6.4 build_status_view(): choosing a view safely

```

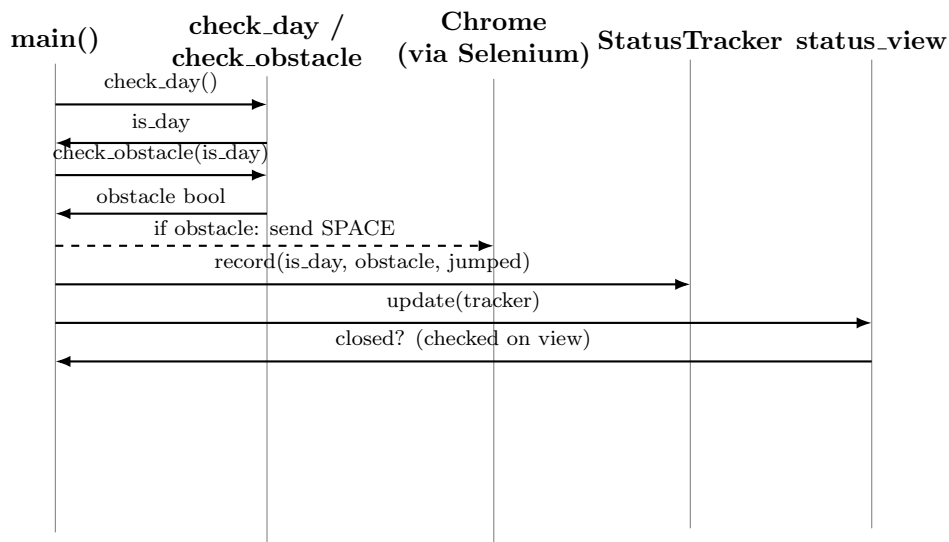
1 def build_status_view():
2     try:
3         return TkStatusView()
4     except Exception as exc:
5         print(f"[status] no display available ({exc}); using console
6             status output")
7         return ConsoleStatusView()

```

Constructing a `tk.Tk()` root window raises an exception if no display is reachable (no `DISPLAY` environment variable on Linux, Tk not installed, running inside a headless CI container, etc.). Rather than letting that exception crash the whole bot, `build_status_view` catches any exception from the attempt and falls back to the console view, so the bot always ends up with *some* working status view and never crashes purely because a GUI could not be created.



6.5 Sequence of one loop iteration, end to end



7 Directory layout

```
dino-game-bot/  
  main.py      detection loop + dashboard  
  requirements.txt  selenium, Pillow, keyboard  
  .env.example   optional CHROMEDRIVER_PATH  
  README.md  
  LICENSE  
  docs/explainers/  this document + the Brief
```

8 Dependencies and their roles

Package	Role
selenium (4.24.0)	Drives the real Chrome browser: navigation, finding elements, sending key events.
Pillow (10.4.0)	Provides ImageGrab, used for the actual screen-region captures the detection logic reads.
keyboard (0.13.5)	Global hotkey detection for the <code>q</code> quit key, read at the OS input-device level rather than through the browser (which is why it needs elevated privileges on Linux).

`tkinter` is not in `requirements.txt` because it ships with the Python standard library (not a separately installed package), though some minimal Python installations omit its native bindings, which is one more reason `build_status_view` treats its absence as a normal, recoverable case rather than an assumed-present dependency.

9 Setup and usage, as documented

```
1 python3 -m venv venv  
2 source venv/bin/activate  
3 pip install -r requirements.txt  
4 python3 main.py
```

Chrome itself must already be installed. Selenium Manager (bundled since Selenium 4.6) downloads and locates a matching ChromeDriver automatically in the common case; `CHROMEDRIVER_PATH` in `.env.example` is only needed to pin a specific driver binary manually. On launch, Chrome opens `chrome://dino/`, is maximized, sent an initial space keypress to start the run, and then the polling loop and status view begin. The user re-measures `OBSTACLE_AREA_COORDINATES` / `GAME_AREA_COORDINATES` for their own screen before relying on it, since both are pinned to one specific resolution and window layout.

10 Documented challenges and design trade-offs

These are drawn directly from the project’s own README “Challenges” section:

- Coordinates are hardcoded to one screen setup, with no logic to derive them from actual window geometry.

- Day/night color inversion requires computing mode before every obstacle check, at the cost of two screen grabs per loop iteration instead of one.
- Some ChromeDriver versions raise a `WebDriverException` on navigating directly to `chrome://dino/` even though the page loads correctly; the code swallows exactly that exception.
- The 10 ms post-jump sleep trades CPU usage for faster reaction time.
- The obstacle box is a binary presence check with no distance/speed estimate, so there is no margin to tune jump timing for taller obstacles or double cacti.
- The original script hardcoded a Windows-only chromedriver path; this version instead defaults to Selenium Manager and makes the path optional.

11 What was verified vs. not (per the README)

Verified in the development environment (a display was available, but no Chrome/live game): the module imports without side effects; `check_day` / `check_obstacle` behave correctly against synthetic in-memory images; `StatusTracker` was driven through a scripted sequence of events and its snapshot state asserted at each step; `TkStatusView` was smoke-tested against a live X display for construction, update, and clean close; the headless fallback was verified by unsetting `DISPLAY` and confirming a `ConsoleStatusView` is returned instead of a crash; `requirements.txt` installs cleanly into a fresh virtual environment.

Not verified: actually launching Chrome and playing a live game; whether the hardcoded coordinates match any screen other than the original one; end-to-end obstacle-clearing reliability; whether the dashboard update introduces visible reaction-time lag against a real, fast game loop.

12 License

This project is licensed under the **PolyForm Strict License 1.0.0**. In plain terms: the software may be viewed and used for permitted purposes (noncommercial use, personal research/study/hobby use, and use by noncommercial organizations such as charities, schools, and public research bodies), but the license does *not* grant any right to redistribute the software or to create and share modified versions of it. It is not the MIT license; there is no permissive reuse or redistribution right here.